Author: Suhas Suresha & Dewang Sultania

SECTION 1

Why AI observability differs from uptime monitoring

- **Infrastructure vs output:** Latency, errors, and uptime still matter, but a fast 200 can hide a confident hallucination when outputs are stochastic and assembled from many dynamic inputs.
- **Silent quality drift:** A prompt revision can leave p95 and error rates flat while turn count and user satisfaction worsen because nothing in classic monitoring measures helpfulness.
- **Pipeline-dependent answers:** Retrieval hits, session truncation, routing and fallback, and guardrail edits all shape the final text; healthy components can combine into a bad response.
- **Token economics:** Cost scales with conversation length, model choice, cache behavior, and prompt bloat in ways CPU-based hosting metrics never capture.
- **Cross-service stories:** One user question touches Session, Data, Guardrails, Model, and Tools; without correlation, debugging becomes timestamp guessing across isolated log streams.
- **Three executive questions:** Finance, product, and platform leads repeatedly ask cost, whether it works, and whether it is improving; observability must answer all three.

SECTION 2

The data model (sessions, traces, spans, generations, scores, logs, metrics)

- **Trace:** The full journey of one user request, carrying input, output, duration, cost, workflow and user metadata.
- **Span:** A timed unit of work inside a trace (session load, search, guardrail pass); nesting produces the waterfall that localizes bottlenecks.
- **Generation:** A specialized span for LLM calls with first-class model, tokens, cost, and time-to-first-token fields so cost and model analytics do not scan every span type.
- **Session:** Groups many traces into a conversation for session-level cost, length, and quality trends.
- **Score:** A named quality judgment on a trace, span, or generation from automated, judge, or human sources; scores often arrive after the user already has the reply.
- **Logs and metrics:** Structured logs explain *why* inside a span; counters and histograms power fleet dashboards; all three tie back to trace_id for drill-down.

SECTION 3

The Observability Service contract and how services report telemetry

- **Receiver, not caller:** Workflows invoke product services; Observability mostly ingests spans, generations, metrics, logs, and scores through a focused gRPC surface.
- **Ingest vs query split:** Hot-path publishing must be asynchronous; slowing `models.chat` to flush telemetry turns observability into an outage multiplier.
- **Buffered client:** Domain services extend a traced base and use an observability client that batches, retries, and caps buffers so brief telemetry outages do not drop user traffic.
- **Automatic instrumentation:** Platform services open `trace_operation` or `trace_generation` around their work; authors call `platform.models.chat` without hand-wrapping every hop.
- **Custom spans:** Application-specific rerankers or external HTTP steps use the same primitives so gaps do not appear between platform boxes.
- **Sensitive reads:** Generation payloads may contain prompts and retrieved text; query access is narrower than ingest.

SECTION 4

Quality scores and cost attribution

- **Heuristic scorers:** Cheap, deterministic checks (required elements, format) run on high traffic as an always-on safety net for structure.
- **LLM-as-judge:** Flexible rubrics for tone and helpfulness cost a model call each time; sample and prefer a different judge model than the one under test.
- **Human annotation:** Ground truth on a slice calibrates automation and catches edge cases scores miss.
- **Per-request dollars:** The Model Service records leaf cost; observability rolls up by team, workflow, and model for finance-friendly drill-down.
- **Cost without quality misleads:** A expensive workflow may be excellent; a cheap one may burn tokens on verbose, unhelpful answers.
- **Budget alerts:** Thresholds and projections warn before limits; enforcement stays in rate limiting, not silent request blocking.

SECTION 5

Experimentation, datasets, offline/online evaluation, and human annotation

- **Passive vs active:** Observability watches; experimentation changes configuration and measures impact on the same trace and score foundation.
- **Targets, not copies:** Prompts and retrieval settings live in their home services; experimentation stores pointers, lifecycle state, and eval history to avoid divergent "v3" copies.
- **Datasets:** Curated cases plus production exports (with review) feed offline runs that repeat stochastic outputs and surface regressions on individual items.
- **Offline evaluation:** Compare prompt or model targets on a fixed dataset before shipping; full-stack harnesses matter when retrieval interacts with the change.
- **Online scoring rules:** Sample live traces, attach scorers in the background, and alert when quality moves while latency does not.
- **Annotation queues:** Route low scores, guardrail flags, or random samples to humans whose labels return into datasets for the next offline cycle.

SECTION 6

A/B testing and the full request picture

- **Sticky assignment:** Hash experiment id with user id so one person stays on one variant for the experiment's life; per-request hashing confounds results.
- **One knob per test:** Change prompt, model, or retrieval—not all three—so a winning metric has a single attributable cause.
- **Tagged traces:** Every production request still records latency, cost, generations, and scores with variant metadata for analysis.
- **Statistical stopping:** Duration, minimum sample size, and confidence on success metrics prevent shipping on intuition or a lucky week.
- **Promotion or revert:** Winners update registry pointers; losers deprecate with zero impact on users who remained on control.
- **Closed loop:** Traffic produces traces and scores; weak scores become dataset cases and hypotheses; offline and online eval plus A/B tests feed the next observability baseline.